channel3d.py reference

Stage 2 of the 3D multipath simulator: 3D path lists to complex channel responses, with a choice of carrier-phase model and optional receive arrays. Reference for every function and class, with four executed examples that use trace3d and channel3d together. Companion to trace3d_reference.pdf. Channel file format version 1.0. Written 2026-10-24.

Contents

Overview 2

The phase model . . . 2

What stage 2 reads from stage 1 3

1. Array helper 3

make_ula3d . . . 3

2. ChannelSet3D and HDF5 3

ChannelSet3D . . . 3

ChannelSet3D.narrowband . . . 4

ChannelSet3D.envelope . . . 4

ChannelSet3D.wideband_gain_db . . . 4

save_channels3d . . . 4

load_channels3d . . . 4

HDF5 layout . . . 5

3. Building channels 5

build_channels3d . . . 5

make_channels3d . . . 5

PHASE_MODES . . . 5

Re-exported from channel2d . . . 6

4. Examples (executed) 6

Example 1: one receiver, both phase modes . . . 6

Example 2: what each phase mode does to spatial correlation . . . 7

Example 3: a four-element array in random-carrier mode . . . 7

Example 4: files, impulse response and the band check . . . 8

Choosing a phase mode and practical notes 8

Verification 9

Overview

What it does. `channel3d.py` reads a `PathSet3D` written by `trace3d.py` and builds the complex channel H(f) of every receiver for one frequency band. Unlike channel2d, it does no material or Fresnel physics: the 3D tracer already stored one complex amplitude per path (spreading, reflections and polarization at the trace frequency). This module applies the band, decides how the carrier phase of each path is set, and can evaluate a receive array.

Where it sits.

Step	Code	File
1. Trace paths (one carrier)	`trace3d.trace_paths3d`	paths .h5 (`save_pathset3d`)
2. Build channels (band, phase mode, array)	`channel3d.build_channels3d` or `make_channels3d`	channels .h5 (`save_channels3d`)
3. Analysis (next stage)	`load_channels3d`, then fits, diversity, ...	figures, tables

Reuse of channel2d. `BandConfig`, `BANDS` and `cir` are imported from `channel2d` and re-exported, so `channel3d.BANDS` is the same band table. Nothing else is shared; the 3D classes and files are separate.

Dependencies. numpy, h5py, `channel2d`, `trace3d` (only its path-file part, so Sionna is not needed here).

The phase model

For receiver point r, array element m and frequency sample n:

```
H_m(f_n) = sum_k  a_k exp(j theta_k) exp(-j 2 pi (f_n - fc) tau_k)
                      * exp(+j 2 pi f_n (d_m . u_k) / C0)

tau_k = L_k / C0        path delay
d_m                     offset of element m from the traced point
u_k                     arrival unit vector (points toward the source)
a_k                     stored amplitude, without propagation phase
```

phase_mode	theta_k	Meaning
`geometric`	-2 pi fc tau_k	Exact path phase. With one antenna the result equals channel2d's model. Nearby receivers are correlated, as in reality.
`random_carrier`	uniform on [0, 2 pi), drawn independently per receiver and path	Delays, amplitudes and array steering stay geometric; only the carrier phase is randomized. It is common to all frequencies and all elements of that receiver.

Consequences.

Topic	Note
Spatial correlation	In random mode neighbouring receiver points are uncorrelated by construction (see example 2). Use the array option for anything spatial.
Array correlation	Comes from the traced arrival directions through the steering term, not from the random phases.
Statistics	With many comparable paths, random phases produce Rayleigh or Rician statistics by construction. What a comparison tests is whether the geometry supplies enough comparable paths.
Robustness	Running both modes and comparing is a useful check of the conclusions.

What stage 2 reads from stage 1

PathSet3D field	Used for
`length`	delay tau = length / C0 (phase across the band and the geometric carrier phase)
`a`	complex amplitude of each path
`aoa`	arrival direction, used for array steering
`offsets`	which paths belong to each receiver
`rx_valid`	invalid receivers get NaN rows; valid receivers with no path get zero rows
`fc_trace`	must equal the band carrier (checked)
`meta`	copied into the channel file as `meta['stage1']`
`order, aod`	not used for H; kept for analysis (for example order to separate the direct path)

What to redo when something changes.

You change	Redo
phase mode, seed, array elements, bandwidth, number of frequency samples	stage 2 only
carrier frequency (band)	stage 1 (re-trace at the new carrier), then stage 2
geometry, materials, obstacles, positions, max_order	stage 1, then stage 2

1. Array helper

`make_ula3d`

```
make_ula3d(n_elem, spacing, axis=(1.0, 0.0, 0.0)) -> (n_elem, 3)
```

Element offsets of a uniform linear array centred on the origin, in metres. Pass the result as the elem argument of build_channels3d.

Parameter	Type	Meaning
`n_elem`	int	Number of elements.
`spacing`	float	Element spacing in metres (for example band.wavelength / 2).
`axis`	3-vector	Array direction; normalized internally.

Notes. The array is evaluated about the traced receiver point with a plane-wave model, so the array length should be small compared with the distance to the nearest reflection point.

2. ChannelSet3D and HDF5

`ChannelSet3D`

```
ChannelSet3D(H, freqs, rx, elem, rx_valid, n_paths, tx, band, meta={})
```

Dataclass holding the channels of one transmitter and R receiver points, each with M array elements, for one band.

Parameter	Type	Meaning
`H`	(R, M, N) complex	Channel per receiver, element and frequency. NaN rows for invalid receivers; zero rows for valid receivers with no path.
`freqs`	(N,)	Absolute frequencies in Hz; index N//2 is the carrier.

Parameter	Type	Meaning
`rx`	(R, 3)	Receiver (array centre) positions.
`elem`	(M, 3)	Element offsets from the receiver position (zeros for a single antenna).
`rx_valid`	(R,) bool	False for receivers the tracer rejected.
`n_paths`	(R,) int	Number of paths used per receiver.
`tx`	(3,)	Transmitter position.
`band`	BandConfig	The frequency plan.
`meta`	dict	phase_mode, seed, trace frequency, stage 1 settings, gain convention, source file.

`ChannelSet3D.narrowband`

```
narrowband() -> (R, M) complex
```

Channel at the carrier frequency (the flat-fading channel), H[..., N // 2].

`ChannelSet3D.envelope`

```
envelope() -> (R, M) float
```

Magnitude of the narrowband channel.

`ChannelSet3D.wideband_gain_db`

```
wideband_gain_db() -> (R, M) float
```

10 log10 of the frequency-averaged |H|^2; minus infinity for shadowed receivers, NaN for invalid ones.

`save_channels3d`

```
save_channels3d(filename, ch, source=None) -> None
```

Writes a ChannelSet3D to HDF5 (gzip level 4). The optional source string records the path file it was built from.

Parameter	Type	Meaning
`filename`	str or path	Output file; overwritten if it exists.
`ch`	ChannelSet3D	The channels.
`source`	str or None	Name of the paths file, stored as an attribute.

`load_channels3d`

```
load_channels3d(filename) -> ChannelSet3D
```

Reads a file written by save_channels3d.

Notes. Raises ValueError for another kind of file (for example a 2D channel file) or a different major format version. meta['source'] is restored from the file.

HDF5 layout

Location	Content
/H	(R, M, N) complex, gzip
/freqs, /elem	(N,) frequencies; (M, 3) element offsets
/rx/xyz, /rx/valid, /rx/n_paths	(R, 3) positions; (R,) validity; (R,) path counts
/tx/xyz	(3,) transmitter position
root attributes	format_version, kind = channels3d, created_utc, band_json, meta_json, source

3. Building channels

build_channels3d

```
build_channels3d(ps, band, phase_mode='random_carrier', elem=None, seed=0,
                 dtype=np.complex128, check_fc=True, progress=False) -> ChannelSet3D
```

Turns a PathSet3D into a ChannelSet3D for one band by evaluating the phase model above for every valid receiver.

Parameter	Type	Meaning
`ps`	PathSet3D	Paths from trace3d, traced at the band's carrier.
`band`	BandConfig	Carrier, bandwidth and number of frequency samples.
`phase_mode`	str	'random_carrier' (default) or 'geometric'. Anything else raises ValueError.
`elem`	(M, 3) or None	Array element offsets; default is one element at the traced point.
`seed`	int	Seed of the random carrier phases (ignored in geometric mode). Phases are drawn for every path of the file at once, so a seed reproduces the same channel whichever receivers are valid. Use different seeds for independent realizations.
`dtype`	complex type	complex128 (default) or complex64 to halve memory.
`check_fc`	bool	Raise ValueError if band.fc differs from the trace frequency. Set False only for deliberate sensitivity checks.
`progress`	bool	Print coarse progress.

Returns. ChannelSet3D with H of shape (R, M, N). Receivers with no valid path get zero rows; invalid receivers get NaN rows.

Notes. Calls ps.validate() first. Warns if the array offsets exceed 10 percent of the shortest path length (plane-wave model inaccurate). Array steering uses the exact frequency of each sample, so wide bands include the small spatial-wideband effect. Delays are applied in double precision before casting to dtype.

make_channels3d

```
make_channels3d(paths_file, band, out_file, **kw) -> ChannelSet3D
```

Convenience pipeline: loads a 3D path file, builds the channels (keyword arguments go to build_channels3d), saves them with the path file name as source, and returns them.

PHASE_MODES

```
PHASE_MODES = ('geometric', 'random_carrier')
```

The accepted phase_mode values, handy for looping over both.

Re-exported from channel2d

Name	What it is
`BandConfig(name, fc, bandwidth, n_sc)`	Frozen dataclass: carrier, bandwidth and number of frequency samples (even). Properties delta_f and wavelength; method freqs() returns fc + (n - n_sc/2) delta_f, so the carrier is index n_sc // 2.
`BANDS`	Preset bands: nr_n78 (3.5 GHz, 100 MHz, 256), wifi_5ghz (5.5 GHz, 80 MHz, 256), upper_midband_13ghz (13 GHz, 400 MHz, 512), nr_n257_28ghz (28 GHz, 400 MHz, 512).
`cir(H, band, nfft=None, window='hann')`	Impulse response by IFFT over the last axis, so it accepts the (R, M, N) array of a ChannelSet3D as well as a single (N,) channel. Returns (t, h); 1 / delta_f is the alias-free window.

The selection helpers of channel2d (row_indices, stride_for, ula_groups, local_normalize) work on the first two coordinates and an array of values, so they can be used with grid rows traced at a single height. Remember that in random mode a row of grid points is not spatially correlated.

4. Examples (executed)

Every example below was run when this document was built, and the green block is its real output. They use a `12 m x 4 m x 3 m` room with a `1 m x 1 m x 1.2 m` box at x = 6 to 7 m, a transmitter at (1, 2, 1.5) and Wi-Fi 5 GHz or n78 bands. The examples call the tracer with the default backend choice, so they use the GPU when one works and the CPU otherwise; each takes from a few to about thirty seconds, mostly Sionna start-up and compilation.

Example 1: one receiver, both phase modes

Trace a room with a low box obstacle, then build the channel of one receiver in each phase mode. The receiver sits in the shadow of the box, so the direct path is blocked (specular-only tracing gives a hard shadow) and the strongest path is a first-order reflection. The two modes differ only in the carrier phase of each path, so the narrowband level differs while the amplitudes and delays are the same.

```
import numpy as np
import trace3d as t3
import channel3d as c3

spec = t3.SceneSpec((12.0, 4.0, 3.0), boxes=((6.0, 1.5, 0.0, 7.0, 2.5, 1.2),))
band = c3.BANDS["wifi_5ghz"]
ps = t3.trace_paths3d(spec, tx=(1.0, 2.0, 1.5), rx_points=[[10.0, 2.0, 1.0]],
                      fc=band.fc, max_order=3)
print("paths:", ps.n_paths()[0], "| orders present:", sorted(set(ps.order.tolist())))
print("line of sight present:", bool((ps.order == 0).any()))
k = np.argmax(np.abs(ps.a))
print(f"strongest path: order {ps.order[k]}, length {ps.length[k]:.2f} m, "
      f"|a| = {abs(ps.a[k]):.2e}")
for mode in c3.PHASE_MODES:
    ch = c3.build_channels3d(ps, band, mode)
    level = 20 * np.log10(ch.envelope()[0, 0])
    print(f"{mode:15s} H shape {ch.H.shape}, |H(fc)| = {level:.1f} dB")

paths: 46 | orders present: [1, 2, 3]
line of sight present: False
strongest path: order 1, length 9.86 m, |a| = 2.98e-04
geometric       H shape (1, 1, 256), |H(fc)| = -66.0 dB
random_carrier  H shape (1, 1, 256), |H(fc)| = -64.7 dB
```

Example 2: what each phase mode does to spatial correlation

A row of 399 receivers only 5 mm apart (about a tenth of a wavelength). With the exact geometric phase, neighbouring points see almost the same channel. With random carrier phases every receiver point gets independent phases, so neighbouring points are uncorrelated. This is why spatial questions (diversity, correlation versus spacing) must use the array option, not a row of grid points, when the random mode is used.

```
import numpy as np
import trace3d as t3
import channel3d as c3

spec = t3.SceneSpec((12.0, 4.0, 3.0), boxes=((6.0, 1.5, 0.0, 7.0, 2.5, 1.2),))
band = c3.BANDS["wifi_5ghz"]
rx, xs, _ = t3.make_rx_grid3d((9.0, 10.99), (2.0, 2.0), 1.0, 0.005)   # 399 points, 5 mm apart
ps = t3.trace_paths3d(spec, (1.0, 2.0, 1.5), rx, band.fc, max_order=3)
print(f"{len(rx)} receivers, step {0.005 / band.wavelength:.2f} wavelengths")
for mode in c3.PHASE_MODES:
    h = c3.build_channels3d(ps, band, mode).narrowband()[:, 0]
    rho = np.mean(h[:-1] * np.conj(h[1:])) / np.mean(np.abs(h) ** 2)
    print(f"{mode:15s} correlation of neighbouring points: |rho| = {abs(rho):.2f}")
```

```
399 receivers, step 0.09 wavelengths
geometric       correlation of neighbouring points: |rho| = 0.93
random_carrier  correlation of neighbouring points: |rho| = 0.03
```

Example 3: a four-element array in random-carrier mode

One trace serves a receive array: the array is evaluated about each traced point with plane-wave steering from the stored arrival directions. Ten seeds give ten independent phase realizations, which are pooled to estimate the element-to-element correlation (each snapshot is normalized to unit power across the elements first). Correlation falls as the element distance grows; the values are set by how widely the traced arrival directions spread.

```
import numpy as np
import trace3d as t3
import channel3d as c3

spec = t3.SceneSpec((12.0, 4.0, 3.0), boxes=((6.0, 1.5, 0.0, 7.0, 2.5, 1.2),))
band = c3.BANDS["wifi_5ghz"]
rx, _, _ = t3.make_rx_grid3d((8.0, 11.0), (0.8, 3.2), 1.0, 0.15)
ps = t3.trace_paths3d(spec, (1.0, 2.0, 1.5), rx, band.fc, max_order=3)
elem = c3.make_ula3d(4, band.wavelength / 2, axis=(0, 1, 0))   # 4 elements, half-wave
H = []
for seed in range(10):                                          # independent realizations
    ch = c3.build_channels3d(ps, band, "random_carrier", elem=elem, seed=seed)
    ok = ch.rx_valid & (ch.n_paths > 0)
    h = ch.narrowband()[ok]
    H.append(h / np.sqrt(np.mean(np.abs(h) ** 2, axis=1, keepdims=True)))   # unit power
H = np.concatenate(H)
R = H.conj().T @ H / len(H)
rho = np.abs(R) / np.sqrt(np.outer(np.diag(R).real, np.diag(R).real))
print(f"H shape {ch.H.shape} -> {len(H)} normalized array snapshots")
print("magnitude of element-to-element correlation:")
print(np.array2string(rho, precision=2, floatmode="fixed"))
```

```
H shape (357, 4, 256) -> 3570 normalized array snapshots
magnitude of element-to-element correlation:
[[1.00 0.40 0.27 0.22]
 [0.40 1.00 0.46 0.26]
 [0.27 0.46 1.00 0.41]
 [0.22 0.26 0.41 1.00]]
```

Example 4: files, impulse response and the band check

The file-based workflow: save the 3D path file, build and save channels with the convenience function, reload them, look at the impulse response, and see the safeguard that refuses a band whose carrier differs from the one the paths were traced at.

```
import numpy as np
import trace3d as t3
import channel3d as c3

spec = t3.SceneSpec((12.0, 4.0, 3.0), boxes=((6.0, 1.5, 0.0, 7.0, 2.5, 1.2),))
band = c3.BANDS["nr_n78"]
rx, _, _ = t3.make_rx_grid3d((9.0, 11.0), (1.0, 3.0), 1.0, 0.5)
ps = t3.trace_paths3d(spec, (1.0, 2.0, 1.5), rx, band.fc, max_order=3)
t3.save_pathset3d("paths_3p5.h5", ps)
c3.make_channels3d("paths_3p5.h5", band, "chan_3p5.h5", phase_mode="geometric")
ch = c3.load_channels3d("chan_3p5.h5")
print("loaded:", ch.H.shape, "| phase mode:", ch.meta["phase_mode"],
      "| source:", ch.meta["source"])
t, h = c3.cir(ch.H[10, 0], band, nfft=2048)
print(f"first path delay {ps.delays()[ps.rx_slice(10)][0] * 1e9:.1f} ns, "
      f"strongest CIR tap at {t[np.argmax(np.abs(h[0]))] * 1e9:.1f} ns")
print(f"wideband gain at receiver 10: {ch.wideband_gain_db()[10, 0]:.1f} dB")
try:
    c3.build_channels3d(ps, c3.BANDS["wifi_5ghz"])
except ValueError as e:
    print("band mismatch ->", str(e)[:70] + "...")
```

```
loaded: (25, 1, 256) | phase mode: geometric | source: paths_3p5.h5
first path delay 29.1 ns, strongest CIR tap at 33.8 ns
wideband gain at receiver 10: -58.5 dB
band mismatch -> Paths were traced at 3.5 GHz but the band carrier is 5.5 GHz; re-trace...
```

Choosing a phase mode and practical notes

Topic	Note
Which mode	Random carrier models the statistical-phase view of a real channel; geometric keeps the exact interference pattern of the idealized scene. Compare both.
Realizations	Change seed for independent draws. Pool them as in example 3 for more samples.
Band changes	A new carrier needs a new trace (materials are evaluated at the trace frequency). Bandwidth and sample count can be changed freely.
Memory	H takes R x M x N x 16 bytes in complex128 (8 in complex64). 10,000 receivers, 4 elements, 512 samples is about 330 MB.
Array model	Plane wave about the traced point: one trace serves every element, but the array must be small against the nearest reflection distance (a warning is raised).
Receiver rows	NaN means invalid receiver, zero means valid with no path. Mask with rx_valid and n_paths > 0 before statistics.
Phase accuracy	Sionna stores single precision. The geometric carrier phase therefore carries about 1e-3 rad of error for paths of a few hundred metres, irrelevant in random mode and small in geometric mode.
Model limits	Specular-only paths with hard shadows, one isotropic vertically polarized antenna at each end, quasi-static, no noise or transmit power. State these with any result.

Verification

The tests are in `test_channel3d.py` and need no Sionna.

Check	Result
Same physics through both pipelines	A 2D trace pushed through channel2d and the same paths converted to a 3D path file give the same H (geometric mode).
Carrier phase and gain convention	A single path gives a * exp(-j 2 pi fc L / C0) at the carrier and a flat magnitude across the band.
Array steering	Element phases for a far source equal the exact distances to each element.
Random mode	Reproducible per seed, different across seeds; envelope of one path independent of the phase; delay structure preserved.
Statistics	Many equal paths give the Rayleigh fourth moment and tail probability; a dominant path gives the Rician fourth moment for the chosen K.
Array correlation	Isotropic arrivals give the complex correlation sin(kd) / (kd) between two elements.
Receivers and errors	NaN and zero rows; wrong phase mode, band mismatch and bad element shapes raise; plane-wave warning; file round trips and wrong-kind files.